RAGDesk

AI Knowledge Base + Support Agent Backend (API-first)

A production-grade portfolio project blueprint for backend engineers in the AI era.
Focus: multi-tenancy, RAG, vector search, queues, streaming, security, observability, and evaluation.

What you will ship

- Document ingestion pipeline (extract → chunk → embed → store).
- Chat API with citations + streaming (SSE).
- Tenant isolation, rate limiting, audit logs.
- OpenTelemetry tracing and RAG evaluation runner.

Generated: February 21, 2026

Version 1.0

Executive summary

RAGDesk is a production-grade backend project that implements a multi-tenant “chat with your data” platform using Retrieval-Augmented Generation (RAG). It is designed to demonstrate job-relevant skills: API design, data modeling, background processing, vector search, streaming, observability, and AI-era security guardrails.

What makes this portfolio piece credible in 2026 hiring loops:

- **RAG over fine-tuning:** Most enterprise assistants ground answers in retrieved documents to reduce hallucinations and provide citations.
- **Vector + relational together:** Postgres with pgvector keeps embeddings alongside source metadata and access controls.
- **LLM security:** Prompt injection and data leakage are treated as first-class risks (mapped to OWASP LLM Top 10).
- **LLMOps:** Observability and evaluation close the loop: you can trace a request, replay a dataset, and measure regressions.

Project goals and non-goals

Goals

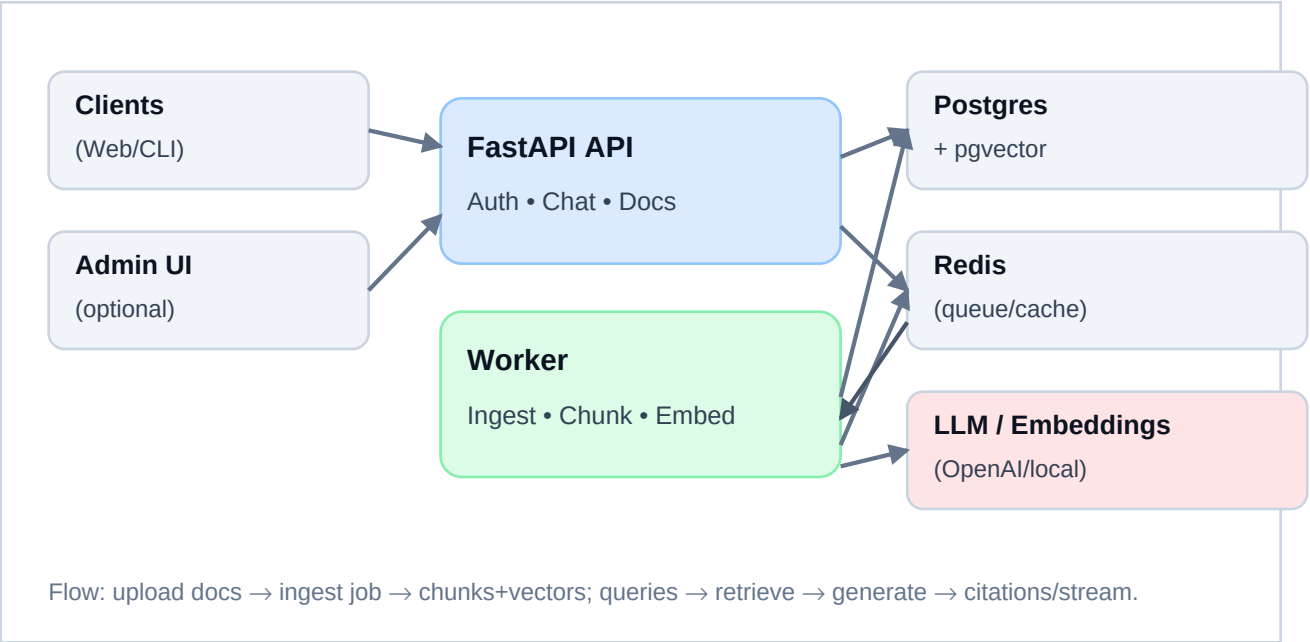
- Ship an API-first backend that can be demoed locally via Docker Compose.
- Support **multi-tenancy** (organizations/knowledge bases) with strict data isolation in both SQL and vector queries.
- Implement an **ingestion pipeline** (extract → chunk → embed → store) using background workers and retries.
- Provide a **chat endpoint** that returns answers with **citations** and optionally streams tokens (SSE).
- Add **observability** (OpenTelemetry tracing + structured logs) and a minimal **evaluation** runner.

Non-goals (for MVP)

- A polished frontend (a CLI or simple admin page is optional).
- Training or fine-tuning models.
- Complex agent toolchains (tool calling can be added later, after solid guardrails).

Architecture at a glance

The system is split into an API service and a worker service. The API handles authentication, document management, and chat requests. The worker performs ingestion jobs (extract/chunk/embed) and writes vectors into Postgres + pgvector. Redis is used for the job queue and caching.



Recommended tech stack

Layer	Choice (recommended)	Why it matters
API	FastAPI + Pydantic	High-performance async API; great OpenAPI docs; strong typing and validation.
DB	Postgres + pgvector	Relational data + vector similarity search; easy joins for citations and access control.
Queue / cache	Redis + RQ/Celery/Arq	Background ingestion, retries, rate limiting counters, short-lived caches.
Migrations	Alembic	Schema evolution you can explain in interviews.
Observability	OpenTelemetry (traces) + JSON logs	Trace retrieval + generation; debug latency and failures.
Testing	pytest + testcontainers	Real integration tests against Postgres/Redis in CI.

Delivery	Docker Compose + GitHub Actions	Reproducible dev env; CI checks (lint/tests) per PR.

Data model (tables)

This schema is intentionally interview-friendly: it demonstrates tenant isolation, traceability for citations, and a feedback loop for evaluation.

- **orgs**(id, name, created_at)
- **users**(id, org_id, email, password_hash, role, created_at)
- **kbs**(id, org_id, name, created_at)
- **documents**(id, org_id, kb_id, source_type, status, created_at, updated_at)
- **chunks**(id, document_id, chunk_index, text, metadata_jsonb, embedding, created_at)
- **chats**(id, org_id, user_id, kb_id, created_at)
- **messages**(id, chat_id, role, content, retrieved_chunk_ids, created_at)
- **feedback**(id, message_id, rating, note, created_at)

Key invariants

- Every table is scoped by **org_id** directly or indirectly (foreign key chain).
- Vector retrieval must always filter by org/kb (never rely on application logic only).
- Messages store **retrieved_chunk_ids** so you can replay and audit answers later.

API surface (MVP)

Keep endpoints small, typed, and predictable. Avoid “magic” agent behavior early; ship reliable retrieval + citations first.

Auth / tenancy

- **POST /auth/register** – create user + org (or join org)
- **POST /auth/login** – JWT access token
- **POST /kbs** – create a knowledge base

Ingestion

- **POST /documents** – upload file or URL; returns document_id
- **GET /documents/{id}** – status and metadata
- **POST /documents/{id}/reindex** – re-chunk + re-embed after config changes

Chat

- **POST /chat** – non-stream answer with citations
- **POST /chat/stream** – Server-Sent Events streaming
- **GET /chats/{id}** – conversation history

Quality loop

- **POST /feedback** – thumbs up/down + optional note
- **POST /eval/run** (later) – batch evaluation on a dataset

Request/response contract (example)

Request	Response

`POST /chat`
`{`
`"kb_id": "kb_123",`
`"chat_id": "chat_456",`
`"message": "What is our refund policy?",`
`"top_k": 6`
`}`
	`200 OK`
	`{`
	`"answer": "Your refund policy allows returns within 30 days ...",`
	`"citations": [`
	`{"chunk_id": "chk_a1", "document_id": "doc_77", "score": 0.83},`
	`{"chunk_id": "chk_b9", "document_id": "doc_52", "score": 0.79}`
	`]`
	`}`

Implementation notes

- Always return **citations** (chunk_id + document_id + score) so the UI can show sources.
- Store retrieved chunk IDs on each assistant message to support **audit and replay**.
- For streaming, prefer **SSE** (simple proxies, easy browser support) with heartbeat and timeouts.
- Enforce tenant filters in the vector query (**org_id**, **kb_id**) to prevent cross-tenant leakage.
- Budget latency: parallelize retrieval and prompt construction; add **timeouts** and circuit breakers around LLM calls.

Security checklist (AI-era)

Treat LLM security as part of backend engineering. Implement these controls early and document them clearly.

Baseline application security

- JWT auth with secure password hashing (argon2/bcrypt).
- RBAC (admin/user) at the org level.
- Rate limiting per user and per org.
- Audit logs for document uploads, reindexing, and chat requests.

LLM-specific risks and mitigations (mapped to OWASP LLM Top 10)

- **Prompt injection:** Treat retrieved documents as untrusted data; enforce a system policy that ignores instructions inside documents.
- **Data leakage:** Tenant filter in retrieval; never return chunks outside org/kb; redact secrets in logs.
- **Insecure output handling:** If tool calling is added later, validate tool arguments with strict schemas.
- **Model DoS / cost blowups:** Enforce per-org quotas, request size limits, timeouts, and circuit breakers.

Observability & LLMOps

Instrumentation turns your demo into a system you can operate. Add traces around retrieval, embedding, and generation, and capture enough metadata to debug failures without leaking sensitive text.

What to instrument (minimum)

- **Trace spans:** auth → embed(query) → vector_search → prompt_build → llm_generate → response_write.
- **Metrics:** p95 latency by endpoint; ingestion throughput; error rate; queue depth.
- **Logs:** request_id, org_id, kb_id, document_id, chunk_ids (not raw chunk text).

Evaluation loop (minimum)

- Create a small test dataset of questions + expected sources.
- Run batch retrieval and measure: hit rate / MRR / groundedness proxies.
- Store eval runs in DB so you can compare before/after changes.

Implementation plan (milestones)

Milestone 0 — Repo + Dev Infra

- FastAPI skeleton: routers/services/repositories; typed settings; OpenAPI tags.
- Docker Compose: API, worker, Postgres+pgvector, Redis.
- Alembic migrations; pytest + basic CI (lint + tests).

Milestone 1 — Auth + Multi-tenancy

- Users, orgs, kbs; JWT login.
- Tenant-scoped dependency in FastAPI; RBAC checks.
- Row-level filters everywhere (including vector queries).

Milestone 2 — Ingestion Pipeline

- POST /documents uploads file; enqueue job.
- Worker: extract text → chunk → embed → store chunks+vectors.
- Document status transitions with retries and dead-letter behavior.

Milestone 3 — Retrieval + Chat

- Embed query; vector search top_k; build prompt with citations.
- Store chats/messages; return answer + citations.
- Add basic caching for embeddings.

Milestone 4 — Streaming + Performance

- SSE streaming endpoint; partial tokens.
- Vector index tuning; paging and metadata filters.
- Load testing script + p95 target.

Milestone 5 — Guardrails + Eval

- OWASP-mapped mitigations documented and implemented.
- Batch eval runner + report output (JSON).
- Tracing and dashboards for demo.

Local quickstart (Docker)

docker-compose.yml (minimal)

```
version: "3.9"

services:
db:
image: pgvector/pgvector:pg16
environment:
POSTGRES_DB: ragdesk
POSTGRES_USER: ragdesk
POSTGRES_PASSWORD: ragdesk
ports:
- "5432:5432"

redis:
image: redis:7-alpine
ports:
- "6379:6379"
```

Enable pgvector extension

```
docker compose up -d
docker exec -it <db_container> psql -U ragdesk -d ragdesk \
-c "CREATE EXTENSION IF NOT EXISTS vector;"
```

Interview talking points

- Explain tenant isolation in both SQL and vector retrieval (why metadata filters matter).
- Discuss chunking tradeoffs (size, overlap, structure-aware splitting) and why it affects recall/precision.
- Show how you traced a slow request with OpenTelemetry spans (where latency comes from).
- Describe your guardrails against prompt injection and data leakage.
- Walk through your ingestion pipeline: idempotency, retries, and document status transitions.

References (selected)

- RAG survey: arXiv 2601.05264 (Retrieval-Augmented Generation for LLMs).
- Microsoft guidance: RAG chunking phase (why chunking is critical).
- pgvector: Postgres extension for vector similarity search.
- OpenTelemetry documentation (observability standard).
- OWASP Top 10 for LLM Applications (security risks and mitigations).
- RAG evaluation: arXiv 2405.07437 (evaluation approaches and metrics).

URLs: arxiv.org/abs/2601.05264 • learn.microsoft.com/azure/architecture/ai-ml/guide/rag/rag-chunking-phase • github.com/pgvector/pgvector • opentelemetry.io/docs • owasp.org/www-project-top-10-for-large-language-model-applications • arxiv.org/abs/2405.07437